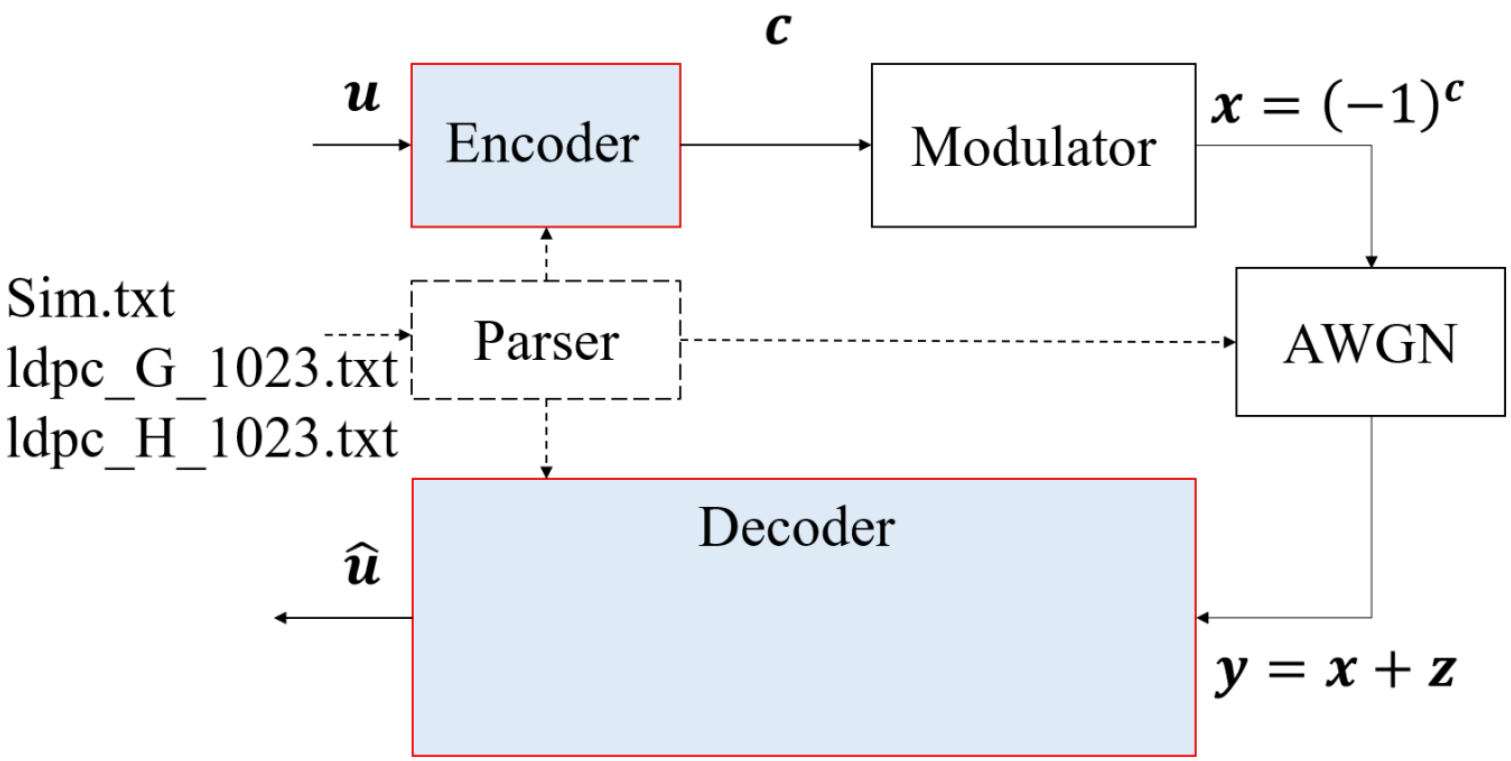
Low-Density Parity-Check Codes

- Consider the $(1023, 781)$ low-density parity-check (LDPC) code with block length $n = 1023$, dimension $k = 781$, row weight $\rho = 32$, and column weight $\gamma = 32$.
- Block Diagram:



Low-Density Parity-Check Codes

- Input Sim.txt

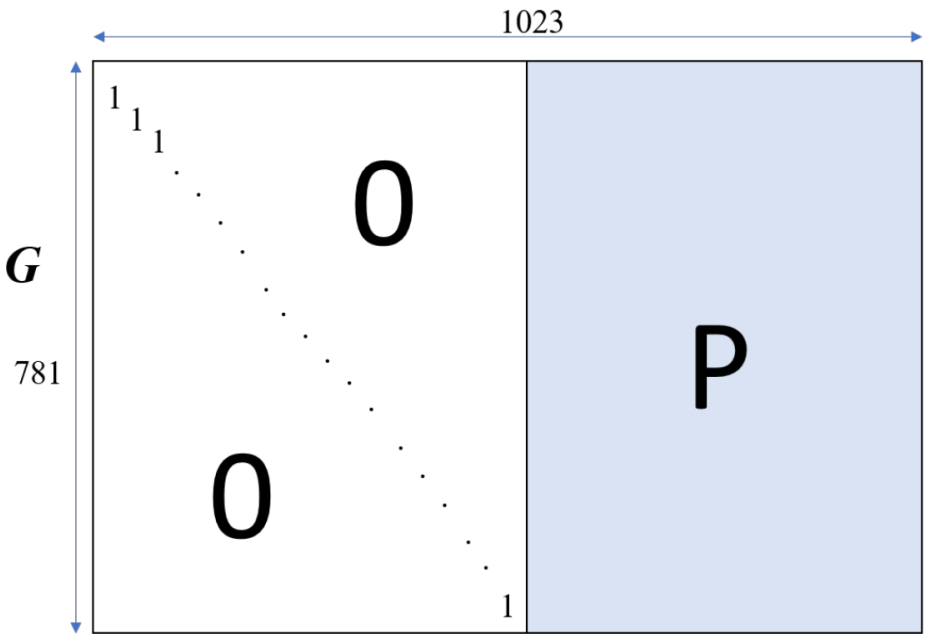
Sim.txt

檔案 編輯 檢視

```
10000 %Number of decoded blocks
100   %Number of iterations
1.0   %SNR: bit signal-to-noise ratio (dB)
-2024 %:Seed: a negative integer
0     %0=>SPA; 1=>MSA; 2=>BF
```

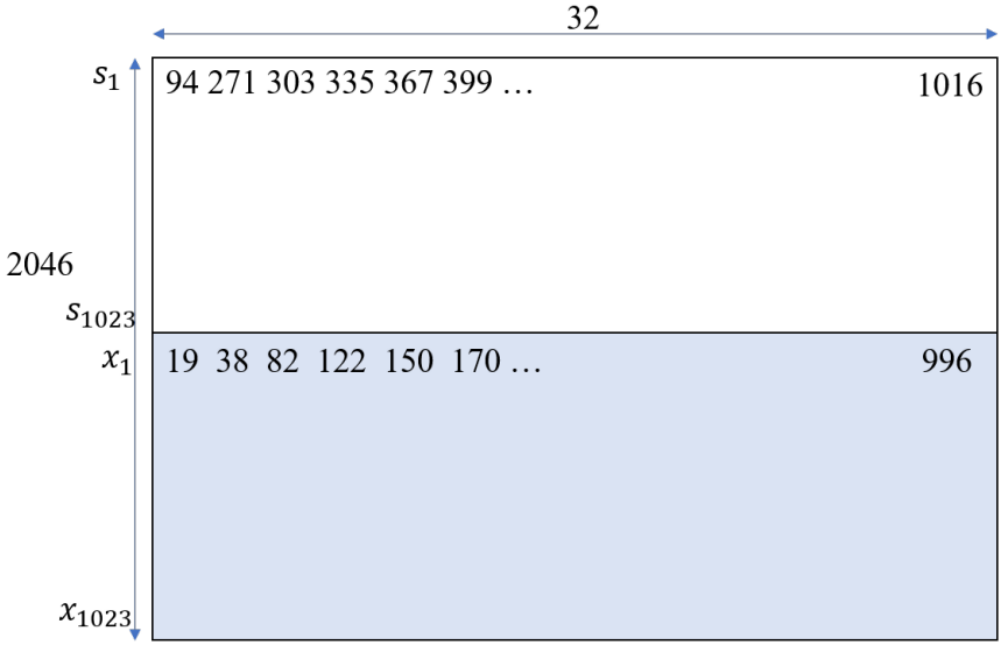
Low-Density Parity-Check Codes

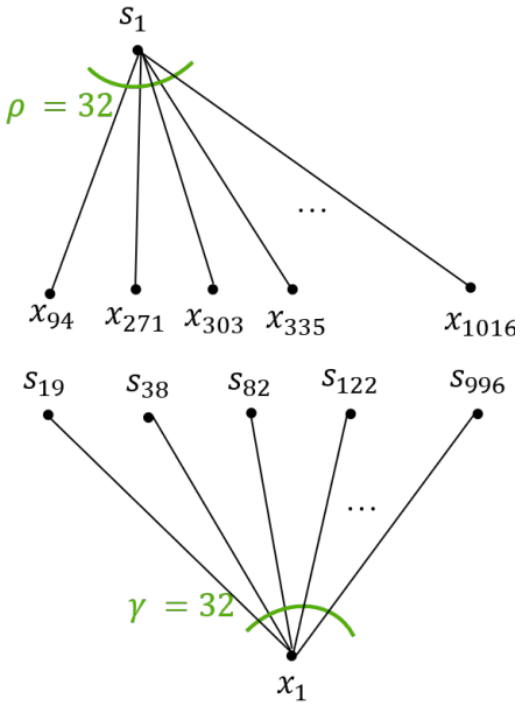
- Input ldpc_G_1023.txt:



Low-Density Parity-Check Codes

- Input ldpc_H_1023.txt:





Encoding of Low-Density Parity-Check Codes

- Use the recursion

$$u_{l+6} = u_{l+1} \oplus u_l, \quad \text{for } l \geq 0$$

with the initial conditions

$$u_0 = 1, \quad u_1 = u_2 = u_3 = u_4 = u_5 = 0$$

to generate $m \cdot k$ information bits where m is the number of test patterns.

- The generated sequence is $100000100001\ldots$ with period 63.

Low-Density Parity-Check Codes

Additive white Gaussian noise (AWGN) channel:

- For a binary code of rate R with BPSK modulation, the noise variance σ^2 is given by

$$\sigma^2 = \left(2R\,\frac{E_b}{N_0}\right)^{-1}.$$

- where E_b/N_0 is the bit signal-to-noise ratio (SNR).
- The code rate is $R = \dfrac{781}{1023}$ here.

Low-Density Parity-Check Codes

- Please use the following pseudo code:

```
#define IA 16807
#define IM 2147483647
#define AM (1.0/IM)
#define IQ 127773
#define IR 2836
#define NTAB 32
#define NDIV (1+(IM-1)/NTAB)
#define EPS 1.2e-7
#define RNMX (1.0-EPS)

main()
{
    ...
    long *idum;
    idum = (long *)malloc(sizeof(long));
    *idum = SEED;  //SEED must be a negative integer.
    ...
    ...
}
```

Low-Density Parity-Check Codes

- Please use **normal()** to output two independent normal random variables, n_1 and n_2.

$$
\begin{aligned}
&\text{normal}(n_1, n_2, \sigma)\\
&\{\\
&\quad \text{do}\{\\
&\qquad x_1 = \text{ran1}(\text{idum});\\
&\qquad x_2 = \text{ran1}(\text{idum});\\
&\qquad x_1 = 2x_1 - 1;\\
&\qquad x_2 = 2x_2 - 1;\\
&\qquad s = x_1^2 + x_2^2;\\
&\quad \}\ \text{while}\ (s \geq 1.0);\\
&\quad n_1 = \sigma x_1 \sqrt{-2 \ln s / s};\\
&\quad n_2 = \sigma x_2 \sqrt{-2 \ln s / s};\\
&\}
\end{aligned}
$$

Low-Density Parity-Check Codes

- Please use **ran1()** to generate a random variable uniformly distributed in the interval $(0, 1)$.

```
ran1(long *idum)
{
    int j;
    long k;
    static long iy=0;
    static long iv[NTAB];
    double temp;
    if (*idum <= 0 || !iy){
       if (-(*idum) < 1) *idum=1;
       else *idum = -(*idum);
       for (j=NTAB+7;j>=0;j--){
           k=(*idum)/IQ;
           *idum=IA*(*idum-k*IQ)-IR*k;
           if (*idum < 0) *idum += IM;
           if (j < NTAB) iv[j] = *idum;
       }
       iy=iv[0];
    }
    k=(*idum)/IQ;
    *idum=IA*(*idum-k*IQ)-IR*k;
    if (*idum < 0) *idum += IM;
    j=iy/NDIV;
    iy=iv[j];
   iv[j] = *idum;
   if ((temp=AM*iy) > RNMX) return RNMX;
   else return temp;
}
```

Decoding of Low-Density Parity-Check Codes

參考數值：

- Sum-Product Algorithm (Maximum 100 iterations):
  - BER=5.4×10^{-2} and BLER=9×10^{-1} at $E_b/N_0 = 2.0$dB.
  - BER=1.29×10^{-3} and BLER=2.45×10^{-2} at $E_b/N_0 = 3.0$dB.
  - BER=1.09×10^{-4} and BLER=1.94×10^{-3} at $E_b/N_0 = 3.4$dB.
- Min-Sum Algorithm (Maximum 100 iterations):
  - BER=9.12×10^{-4} and BLER=9.62×10^{-3} at $E_b/N_0 = 4.2$dB.